

Relatório de Análise

Ollama Chat & Benchmark Laboratory (Blazor Server, .NET 10)

Aplicação: OllamaFluentUIChat | Data: 10 de agosto de 2026

Este documento é o resultado de uma análise estática da base de código, cobrindo os pontos fortes, os defeitos técnicos críticos, a dívida técnica, questões funcionais/UX e um plano de melhoria em quatro fases. Todas as referências usam o formato `ficheiro:linha`.

Índice

- Pontos fortes
- Bugs técnicos críticos
- Dívida técnica / código morto
- Funcional / Experiência de utilizador
- Plano de melhoria proposto (Fases 1 a 4)

1. Pontos fortes

- **Conceito diferenciado.** A ideia de "laboratório de benchmark" — medir automaticamente tokens/s, tempo de carga e de processamento em cada resposta, cruzar com avaliação de juízes externos (Gemini/OpenRouter) e gerar análise local — é sólida e bem documentada (README PT/EN, `Funcionalidades.md`, `schema.sql`).
- **Localização madura.** `SharedResources.resx` / `.en.resx` com 334 chaves cada, paridade total, e ~99% dos textos da interface passam por `L[...]`. Pouco comum.
- **SQL seguro.** Todo o acesso a dados usa Dapper com parâmetros `@`; não foi encontrada nenhuma SQL injection.
- **ChatMeasureTemperature** (`Services/ChatMeasureTemperature.cs`) — um dos melhores ficheiros do projeto: heurística lexical PT/EN para temperatura, deteção de código, densidade de perguntas; bem comentado, testável e sem dependências.
- **Arquitetura em camadas** com interfaces (`Repositories/Services` e distinção DTO vs Entities) — boa intenção que facilita a correção da dívida.
- **Acessibilidade parcial boa** — `aria-expanded` nos cabeçalhos colapsáveis, `@onclick:stopPropagation` nos cards, diálogos com `AriaLabel`.
- **Streaming de chat robusto** na via ativa (`SendMessage`, `Components/Pages/Chat.razor.cs:208`) — timeout alargado, cancelamento com descarga da VRAM e gestão de contexto dinâmico.

2. Bugs técnicos críticos

#	Problema	Evidência
1	~700 linhas de código duplicado: três implementações de envio de mensagem (<code>SendMessage</code> ativa; <code>SendMessageWithOllamaSharpAsync</code> e <code>SendMessageWithOllamaFastAsync</code> mortas).	<code>Chat.razor.cs:208, 868, 1094</code>
2	Serilog grava cada log 2 vezes — sink definido no JSON e novamente no <code>Program.cs</code> .	<code>Program.cs:32-39</code> + <code>appsettings.json:19-29</code>
3	ON DELETE CASCADE não funciona — <code>PRAGMA foreign_keys</code> nunca é ativado; apagar um prompt deixa <code>Respostas</code> órfãs.	<code>DapperContext.cs:25</code> vs <code>Docs/schema.sql:5</code>
4	Agrupamento por prompt quebrado — <code>BenchmarkResponseEvaluationAsync</code> nunca seleciona <code>R.PromptId</code> , logo <code>PromptId</code> é sempre 0.	<code>BenchmarkRepository.cs:368-385</code>
5	Crash na análise local quando todos os <code>*FactualRating</code> são NULL — <code>.First()</code> lança exceção.	<code>LocalAnalysisService.cs:92-102</code>
6	Check de VRAM usa WMI <code>AdapterRAM (uint32)</code> que devolve 0 em GPUs > 4 GiB, ignorando o caminho <code>nvidia-smi</code> já implementado; além disso <code>nvidia-smi</code> bloqueia porque <code>ReadToEnd()</code> precede <code>WaitForExit</code> .	<code>OllamaGpuService.cs:170, 114-115</code>
7	<code>/culture-reload</code> — XSS refletido / open redirect: <code>redirectUri</code> interpolado cru em HTML/JS.	<code>Program.cs:114-120</code>
8	Rate-limit dos juizes consumido mesmo em falha e validado antes das chaves da API.	<code>AutomatedJudgeService.cs:102-103, 121-129</code>
9	Concorrência SQLite: <code>VACUUM</code> após cada delete + <code>batchSize:1</code> + sink duplicado; exceções "database is locked" são engolidas → perda silenciosa de dados.	<code>LogRepository.cs:38,</code> <code>BenchmarkRepository.cs:68-71,</code> <code>Chat.razor.cs:509-512</code>
10	<code>HttpClient</code> compartilhado com <code>DefaultRequestHeaders</code> mutado nas pesquisas web — condição de corrida com a descarga de modelo em background.	<code>Chat.razor.cs:724-730, 552, 690</code>
11	Erros mascarados: <code>TranslationService</code> lança <code>InvalidOperationException("Sem informação de modelos")</code> para qualquer falha, sem exceção interior.	<code>TranslationService.cs:49-53</code>

3. Dívida técnica / código morto

- **Páginas mortas e não ligadas no menu:** `Settings.razor` (lista de modelos fake hardcoded), `Settings2.razor` (refactor abandonado a meio), `AnaliseBenchmark.razor` (substituída por `/benchmark-evaluations`).
- **`ConversationRepository`** chama stored procedures SQL Server (`usp_Conversation_*`) contra SQLite — rebentaria se fosse registado em DI.

- **DTOs/serviços nunca usados:** `TranslateClientService`, `ChatMessageDto`, `ChatMessageModel`, `HistoryEntry`, `ConversationItem`, `LanguageDetectionResult`, `TranslateRequest/ResponseDto`, `Entities\ChatMessage`, `QualityAnalysis.resx` inteiro.
- `TranslatePromptTemplate.TranslationPrompt` é um duplicado morto do `translation-prompt.txt`.
- `PromptTemplateProvider._cache` nunca é preenchido — I/O a disco em cada chamada.
- `SystemPromptService` registado em DI mas nunca injetado (o Chat lê o ficheiro diretamente).
- **RAG web morto:** `SearchWebContext_DuckDuckGo_Async` e `SearchWebContext_Wikipedia_Async` — funcionalidade completa, mas sem botão/ligação.
- **Interop JS roto:** `BenchmarkAnalysisViewer.razor:144` chama `scrollToBottom` inexistente; `Chat.razor.cs:597` chama `chatInput.handleKey` inexistente.
- `SharpToken` na csproj, mas a estimativa de tokens é `len/4` (`Chat.razor.cs:646-650`).

4. Funcional / Experiência de utilizador

- **Histórico fragmentado:** localStorage do browser (via JS) e `BenchmarkRepository` na BD — dois sistemas de memória de conversas.
- **Cores hardcoded** (116 ocorrências hex/rgba) quebram o tema escuro; o próprio `NavMenu.razor:53-56` usa `#f0f0f0` invisível no dark mode.
- `FluentDesignTheme` **duplicado** (MainLayout + Settings + Settings2 + ModelosOllama).
- **ModelosOllama** mostra "Nenhum modelo detetado" quando na verdade houve erro.
- **Botões só-ícone sem** `aria-label` (eliminar logs, downloads de gráficos).
- `appsettings.json` COM `"DetailedErrors": true` e `AllowedHosts: "*" .`
- Chaves de API em texto simples em `appsettings.Local.json`; a chave Gemini viaja no query string da URL.
- **Ollama base URL hardcoded em 10+ sítios** (`localhost:11434`); zero configuração centralizada.

5. Plano de melhoria proposto

Fase 1 — Estabilizar (bugs, sem mudança de comportamento)

- 1. Remover as duas implementações mortas de `SendMessage` e extrair a lógica de streaming, histórico e benchmark de `Chat.Razor.cs` para um serviço injetável.
- 2. Corrigir o sink Serilog duplicado (deixar apenas o do `Program.cs`).
- 3. Ativar `PRAGMA foreign_keys=ON + busy_timeout` no `DapperContext`; substituir `VACUUM` por `DELETE + PRAGMA optimize`.
- 4. Corrigir as projeções `R.PromptId` e `TempoProcessamento`; guarda contra `.First()` no `LocalAnalysisService`.
- 5. `CheckGpuCompatibility` deve usar `GetUsableVramBytes()`; `nvidia-smi` async com `WaitForExit` antes da leitura.
- 6. Sanitizar `redirectUri` no `/culture-reload`; chaves API para user-secrets; `DetailedErrors=false`.
- 7. `AutomatedJudgeService`: validar chaves primeiro e só marcar timestamps após sucesso.

Fase 2 — Consolidar (arquitetura)

- 8. Centralizar `Ollama:BaseUrl`, opções de chat, overhead de VRAM e contexto default no `appsettings.json` (via `IOptions`).
- 9. Remover dead code: `Settings2`, `AnaliseBenchmark`, `ConversationRepository`, DTOs mortos, `TranslateClientService`, cache morto do `PromptTemplateProvider`.
- 10. Fundir `Settings/Settings2` numa página real (listar modelos reais do Ollama, gerir favoritos).
- 11. Migrations incrementais idempotentes (scripts SQL por versão executados no arranque) no lugar do `EnsureRecommendationColumns` ad-hoc.
- 12. Remover `ollama_benchmark.db` do git e gerá-la no arranque.

Fase 3 — Features novas (maior valor)

- 13. Ativar RAG web no chat (a lógica DuckDuckGo/Wikipedia já existe) com botão/toggle e citações.
- 14. Suporte a modelos de reasoning (deepseek-r1, qwq) — mostrar `reasoning_content` separadamente.
- 15. Persistência real de conversas na BD (deixar de depender de `localStorage`) com export/import JSON.
- 16. Dashboard agregado de métricas (gráficos por modelo ao longo do tempo) em vez de só tabelas.
- 17. Testes unitários (`ChatMeasureTemperature`, `EvaluationParser`, `LocalAnalysisService`, repositórios com SQLite in-memory) + CI com GitHub Actions (`dotnet build`).

Fase 4 — UX

- 18. Substituir cores hardcoded por design tokens FluentUI; `aria-label` em botões-ícone; estados de erro reais nas páginas.